
Université Abdelmalek Essaâdi

Faculté des Sciences et Techniques de Tanger

Département Génie Informatique

Master en Intelligence Artificielle et Sciences des Données

Demi-Module : Traitement du Langage Naturel (NLP)

Rapport de Projet

*Conception et mise en œuvre d'un système de génération
automatique de règles YARA à l'aide du NLP et du RAG*



Réalisée par :

Afailal Trebak Ghizlane

Ibn Ibrahym Meriem

El Boumashouli Achraf

El Mansouri Aya

El Rhorba Aya

Encadrée par :

Pr. Ikram Benabdelouahab

Année Universitaire 2025/2026

Résumé

La détection des logiciels malveillants constitue l'un des défis les plus pressants de la cybersécurité contemporaine. Les analystes en sécurité s'appuient traditionnellement sur des règles YARA — un langage de description de patterns permettant d'identifier des fichiers malveillants sur la base de leurs caractéristiques statiques et comportementales — pour automatiser la détection des menaces. Cependant, la rédaction manuelle de ces règles exige une expertise approfondie et représente un investissement en temps considérable, ce qui ralentit la réponse aux menaces émergentes.

Ce projet de fin d'études propose un système intelligent capable de **générer automatiquement des règles YARA** à partir de descriptions en langage naturel décrivant des comportements malveillants. Notre approche repose sur une architecture à trois couches complémentaires : un pipeline de **traitement automatique du langage naturel (NLP)** chargé de construire et d'enrichir une base de connaissances spécialisée en cybersécurité, un module de **Génération Augmentée par Récupération (RAG)** implémentant des stratégies de retrieval hybride pour identifier les contextes les plus pertinents, et un composant de **génération par grands modèles de langage (LLM)** transformant ces contextes en règles YARA syntaxiquement valides et sémantiquement cohérentes.

La base de connaissances construite contient **3 046 enregistrements** issus de 2 850 documents bruts — rapports d'analyse de malwares, fichiers de règles YARA et rapports de renseignement sur les menaces — enrichis par **431 enregistrements synthétiques** produits selon quatre stratégies d'augmentation. Le module RAG intègre trois architectures — classique, hybride et agentique — combinant la recherche sémantique dense (FAISS) et la recherche par mots-clés creuse (BM25). Trois modèles de langage de taille variable sont évalués : Qwen2.5-0.5B-Instruct, Flan-T5-base et Mistral-7B-Instruct.

Mots-clés : YARA, cybersécurité, NLP, RAG, LLM, génération de règles, détection de malwares, FAISS, BM25, Qwen, Mistral, intelligence artificielle, base de connaissances.

Abstract

Malware detection remains one of the most pressing challenges in contemporary cybersecurity. Security analysts traditionally rely on YARA rules — a pattern-matching language for identifying malicious files based on their static and behavioral characteristics — to automate threat detection. However, crafting these rules manually requires deep domain expertise and significant time investment, slowing response to emerging threats.

This final-year project presents an intelligent system capable of **automatically generating YARA rules** from natural language descriptions of malicious behaviors. Our approach rests on a three-layer complementary architecture: a **Natural Language Processing (NLP)** pipeline responsible for constructing and enriching a specialized cybersecurity knowledge base; a **Retrieval-Augmented Generation (RAG)** module implementing hybrid retrieval strategies to identify the most relevant contexts; and a **Large Language Model (LLM)** generation component that transforms these contexts into syntactically valid and semantically coherent YARA rules.

The constructed knowledge base contains **3,046 records** derived from 2,850 raw documents — malware analysis reports, YARA rule files, and threat intelligence reports — enriched with **431 synthetic records** produced according to four augmentation strategies. The RAG module integrates three architectures — classic, hybrid, and agentic — combining dense semantic search (FAISS) and sparse keyword search (BM25). Three language models of varying sizes are evaluated: Qwen2.5-0.5B-Instruct, Flan-T5-base, and Mistral-7B-Instruct.

Keywords: YARA, cybersecurity, NLP, RAG, LLM, rule generation, malware detection, FAISS, BM25, Qwen, Mistral, artificial intelligence, knowledge base.

Contents

Résumé	1
Abstract	2
Liste des abréviations	8
1 Introduction générale	9
1.1 Contexte et motivation	9
1.2 Problématique	10
1.3 Objectifs du projet	10
1.4 Contributions	11
1.5 Organisation du rapport	11
2 État de l’art	12
2.1 Les règles YARA en cybersécurité	12
2.1.1 Syntaxe et structure d’une règle YARA	12
2.1.2 Enjeux de la génération manuelle de règles	12
2.2 Traitement automatique du langage naturel pour la cybersécurité	13
2.2.1 Extraction d’informations à partir de rapports de menaces	13
2.2.2 Limites des approches purement extractives	13
2.3 Génération Augmentée par Récupération (RAG)	13
2.3.1 Principe et architecture générale	14
2.3.2 Retrieval dense vs. retrieval creux	14
2.3.3 RAG agentique	15
2.4 Grands modèles de langage pour la génération de code	15
2.4.1 Capacités génératives des LLM	15
2.4.2 Quantification et modèles légers	15
3 Pipeline NLP et construction de la base de connaissances	16
3.1 Vue d’ensemble de l’architecture NLP	16
3.2 Chargement et extraction des documents	17
3.2.1 Diversité des formats sources	17
3.2.2 Adaptation selon le type de source	18
3.3 Nettoyage et normalisation du texte	18
3.3.1 Artefacts PDF et bruit textuel	18
3.3.2 Conservation des informations sémantiques	18

3.4	Extraction d'entités par NLP	19
3.4.1	Architecture de l'extracteur d'entités	19
3.4.2	Entités extraites	19
3.5	Construction des records structurés	20
3.5.1	Schéma JSON normalisé	20
3.5.2	Système d'identifiants typologiques	20
3.6	Génération du champ d'embedding	20
3.6.1	Motivation d'un champ dédié	20
3.6.2	Construction du champ	21
3.7	Filtrage qualité	21
3.7.1	Critères de scoring	21
3.7.2	Corrections automatiques appliquées	22
3.8	Augmentation synthétique	22
3.8.1	Motivations	22
3.8.2	Les quatre stratégies d'augmentation	22
3.9	Statistiques du dataset final	23
4	Architecture RAG et stratégies de retrieval	24
4.1	Position du module RAG dans l'architecture globale	24
4.2	Indexation et gestion des index	25
4.2.1	Construction des index	25
4.2.2	Gestion de l'état des index	26
4.3	Retrieval dense par FAISS	26
4.4	Retrieval creux par BM25	26
4.5	Retrieval hybride par fusion de scores	26
4.6	Architecture agentique	27
4.6.1	QueryAnalyzer — Analyse de la requête	27
4.6.2	RetrievalAgent — Sélection adaptative de la stratégie	27
4.6.3	ValidationAgent — Contrôle qualité et boucle de correction	27
5	Génération par grands modèles de langage	29
5.1	Ingénierie des prompts	29
5.1.1	Structure générale du prompt	29
5.1.2	Mode few-shot	29
5.2	Les modèles LLM intégrés	30
5.2.1	Qwen2.5-0.5B-Instruct	30
5.2.2	Flan-T5-base	30
5.2.3	Mistral-7B-Instruct	30
5.3	Post-traitement de la sortie	30

5.4	Génération d'explications.....	31
6	Évaluation et résultats.....	32
6.1	Protocole d'évaluation	32
6.1.1	Dimensions d'évaluation	32
6.1.2	Métriques utilisées	33
6.2	Analyse des résultats par architecture RAG	33
6.2.1	Modes classique, hybride et agentique	33
6.2.2	Comparaison des modèles LLM	34
6.2.3	Tableau comparatif des performances	34
6.3	Limites identifiées	35
6.3.1	Valeurs de <code>filesize</code> inventées.....	36
6.3.2	Scores BLEU intrinsèquement faibles	36
6.3.3	Répétition de documents dans les petits datasets	36
7	Architecture logicielle et interface utilisateur.....	37
7.1	API publique	37
7.2	Structure du projet.....	37
7.3	Interface utilisateur.....	38
7.3.1	Page de génération de règles YARA	38
7.3.2	Résultat de génération et sources récupérées.....	39
7.3.3	Recherche dans la base de connaissances.....	39
7.3.4	Statistiques de la base de connaissances	40
7.3.5	Indexation de documents PDF	40
8	Conclusion et perspectives.....	42
8.1	Bilan du projet	42
8.2	Perspectives	42
8.2.1	Amélioration de la base de connaissances	42
8.2.2	Fine-tuning des LLM sur des données YARA	43
8.2.3	Validation exécutable des règles	43
8.2.4	Apprentissage par renforcement depuis le retour humain	43
8.2.5	Extension à d'autres langages de détection	43
	Références bibliographiques	44
A	Exemple complet de record de la base de connaissances	46
B	Exemple de règle YARA générée par le système.....	47
C	Guide d'utilisation de l'API	48

List of Figures

2.1	Architecture générique d'un système RAG	14
3.1	Architecture du pipeline NLP — du document brut au dataset structuré	17
4.1	Architecture détaillée du module RAG	25
4.2	Boucle de contrôle qualité de l'architecture agentique	28
7.1	Interface de génération — sélection du mode RAG	38
7.2	Interface de génération — sélection du modèle LLM	39
7.3	Résultat de génération avec indicateurs de qualité et sources récupérées	39
7.4	Interface de recherche dans la base de connaissances	40
7.5	Tableau de bord statistique — distribution des types et familles de mal- wares	40
7.6	Interface d'upload et d'indexation de documents PDF	41

List of Tables

3.1	Description des champs du schéma JSON de la base de connaissances .	20
3.2	Critères de scoring des records de la base de connaissances	21
3.3	Statistiques globales du dataset de production	23
3.4	Distribution du dataset par type de malware	23
5.1	Comparaison des modèles LLM intégrés	30
6.1	Métriques d'évaluation du système	33
6.2	Performances comparatives des approches RAG avec Flan-T5, Qwen2.5 et Mistral-7B	34

Liste des abréviations

Sigle	Signification
NLP	Natural Language Processing — Traitement Automatique du Langage Naturel
RAG	Retrieval-Augmented Generation — Génération Augmentée par Récupération
LLM	Large Language Model — Grand Modèle de Langage
YARA	Yet Another Ridiculous Acronym
IoC	Indicator of Compromise — Indicateur de Compromission
TTP	Tactics, Techniques and Procedures
APT	Advanced Persistent Threat
FAISS	Facebook AI Similarity Search
BM25	Best Match 25 (algorithme de scoring textuel)
BLEU	Bilingual Evaluation Understudy
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
MRR	Mean Reciprocal Rank
PDF	Portable Document Format
JSON	JavaScript Object Notation
API	Application Programming Interface
MITRE	MITRE ATT&CK Framework
AES	Advanced Encryption Standard
GPU	Graphics Processing Unit
VRAM	Video Random Access Memory

Chapter 1

Introduction générale

1.1 Contexte et motivation

La révolution numérique des deux dernières décennies a profondément reconfiguré le paysage de la sécurité informatique. L’explosion du nombre de logiciels malveillants — désormais comptés en centaines de millions de variantes actives — confronte les organisations à une menace évolutive, polymorphe et toujours plus sophistiquée. Face à cette réalité, les équipes de sécurité ne peuvent plus s’appuyer uniquement sur des mécanismes de détection basés sur des signatures statiques, qui s’avèrent systématiquement dépassés par les nouvelles variantes de malwares conçues précisément pour les contourner.

Dans ce contexte, les **règles YARA** occupent une place centrale dans l’arsenal des analystes en sécurité. Développé initialement par Victor Alvarez chez VirusTotal, YARA est un langage de description de patterns conçu pour identifier des fichiers ou processus malveillants à partir de leurs caractéristiques syntaxiques, structurelles et comportementales. Une règle YARA peut décrire des chaînes de caractères suspectes, des séquences d’octets caractéristiques, des structures de fichiers exécutables particulières, ou encore des conditions complexes combinant plusieurs de ces éléments. Elle constitue ainsi un outil de détection à la fois précis, portable et directement opérationnel dans des environnements de production.

Cependant, la rédaction de règles YARA efficaces représente un exercice exigeant. Elle suppose une maîtrise approfondie du format des fichiers binaires, une connaissance intime des familles de malwares ciblées, ainsi qu’une expérience pratique des techniques d’analyse statique et dynamique. Dans un contexte où les nouvelles menaces apparaissent quotidiennement et où la rapidité de réponse est déterminante, la dépendance à l’expertise humaine manuelle crée un goulot d’étranglement opérationnel significatif.

C’est précisément à cette problématique que le présent projet entend répondre, en exploitant les avancées récentes dans le domaine de l’intelligence artificielle appliquée au traitement du langage naturel.

1.2 Problématique

La question centrale que soulève ce projet peut se formuler de la manière suivante : *est-il possible de concevoir un système capable de générer automatiquement des règles YARA syntaxiquement valides et sémantiquement pertinentes, à partir de descriptions textuelles en langage naturel décrivant des comportements de logiciels malveillants ?*

Cette problématique soulève plusieurs défis interdépendants :

- **Hétérogénéité des sources de connaissances** : les informations sur les malwares sont disséminées dans des rapports d'analyse au format PDF, des bases de données structurées, des dépôts de règles YARA et des rapports de renseignement, sous des formats très variés.
- **Complexité de la syntaxe YARA** : un grand modèle de langage généraliste ne dispose pas naturellement d'une connaissance suffisante des contraintes syntaxiques spécifiques du langage YARA ni des bonnes pratiques de rédaction de règles en cybersécurité.
- **Risque d'hallucination** : les LLM ont tendance à générer des constructions syntaxiques inexistantes ou des fonctions fictives, ce qui invalide immédiatement la règle produite.
- **Ambiguïté des descriptions en langage naturel** : une même description peut correspondre à des familles de malwares très différentes, et la pertinence des exemples fournis au modèle conditionne directement la qualité de la sortie.

1.3 Objectifs du projet

Le présent projet poursuit les objectifs suivants :

1. Construire une **base de connaissances spécialisée** en cybersécurité à partir de sources hétérogènes, enrichie par des données synthétiques.
2. Développer un **pipeline NLP** robuste pour l'extraction, la normalisation et la structuration des informations pertinentes issues de documents bruts.
3. Implémenter un **module RAG** combinant recherche sémantique dense et recherche par mots-clés creuse, avec une architecture agentique capable d'auto-correction.
4. Intégrer et comparer plusieurs **grands modèles de langage** pour la génération de règles YARA.
5. Concevoir un **protocole d'évaluation** rigoureux mobilisant des métriques textuelles, syntaxiques et sémantiques.

6. Offrir une **interface utilisateur** permettant à un analyste non spécialiste en IA d'interagir avec le système de manière intuitive.

1.4 Contributions

Les principales contributions scientifiques et techniques de ce travail sont les suivantes :

- La conception et la mise en œuvre d'un **pipeline NLP multi-étapes** comprenant le chargement multi-format, le nettoyage, l'extraction d'entités nommées par dictionnaires et expressions régulières, la construction de records structurés, et l'augmentation synthétique selon quatre stratégies complémentaires.
- La construction d'un **dataset de 3 046 enregistrements** structurés, couvrant dix catégories de malwares, optimisés pour la vectorisation sémantique via un champ `embedding_text` spécifiquement conçu.
- L'implémentation d'une **architecture RAG agentique** combinant trois agents spécialisés (analyse de requête, décision de retrieval, validation) avec une boucle de rétroaction permettant jusqu'à deux itérations de correction.
- Une **évaluation comparative multi-modèles et multi-architectures** sur 11 métriques couvrant la qualité textuelle, syntaxique YARA, sémantique, et la détection des hallucinations.

1.5 Organisation du rapport

Ce rapport est structuré en six chapitres. Le chapitre 2 présente l'état de l'art dans les domaines concernés. Le chapitre 3 détaille le pipeline NLP et la construction de la base de connaissances. Le chapitre 4 décrit l'architecture RAG et les stratégies de retrieval. Le chapitre 5 traite de la génération par LLM et de l'ingénierie des prompts. Le chapitre 6 présente les résultats d'évaluation et leur analyse. Enfin, le chapitre 8 conclut et trace des perspectives d'évolution.

Chapter 2

État de l'art

2.1 Les règles YARA en cybersécurité

2.1.1 Syntaxe et structure d'une règle YARA

Une règle YARA se compose de trois sections principales : la section **meta** : contenant des métadonnées descriptives, la section **strings** : définissant les patterns à rechercher, et la section **condition** : exprimant la logique de détection. La règle suivante illustre la détection du ransomware WannaCry :

```
1 rule WannaCry_Ransomware {
2     meta:
3         description = "Detects WannaCry ransomware"
4         author      = "Security Team"
5         date        = "2017-05-12"
6     strings:
7         $s1 = "tasksche.exe"  ascii wide nocase
8         $s2 = "mssecsvc.exe"  ascii wide nocase
9         $s3 = ".WNCRY"        ascii wide nocase
10        $s4 = "WannaDecryptor" ascii wide nocase
11        $h1 = { 45 52 4E 41 4C 42 4C 55 45 } // "ETERNALBLUE"
12    condition:
13        filesize < 5MB and
14        3 of ($s*) or $h1
15 }
```

Listing 2.1: Exemple de règle YARA pour la détection de WannaCry

La puissance de YARA réside dans la flexibilité de ses conditions, qui peuvent référencer non seulement la présence de chaînes mais aussi des propriétés structurales du fichier (taille, entropie, en-tête PE), des patterns hexadécimaux, ou encore des expressions régulières.

2.1.2 Enjeux de la génération manuelle de règles

La rédaction efficace de règles YARA constitue un artisanat à part entière. Elle suppose la maîtrise simultanée de plusieurs domaines : l'analyse statique de fichiers binaires, la

connaissance des familles de malwares et de leurs comportements caractéristiques, la compréhension des structures de fichiers exécutables (PE, ELF, Mach-O), et la maîtrise des bonnes pratiques permettant de limiter les faux positifs. Dans un contexte où de nouvelles variantes de malwares apparaissent quotidiennement, la capacité à produire rapidement des règles de qualité représente un avantage opérationnel déterminant — que la génération automatisée peut contribuer à démocratiser.

2.2 Traitement automatique du langage naturel pour la cybersécurité

2.2.1 Extraction d'informations à partir de rapports de menaces

L'application du traitement automatique du langage naturel à la cybersécurité constitue un domaine de recherche en pleine expansion. Les travaux pionniers dans ce domaine ont exploré l'extraction automatique d'indicateurs de compromission (IoC) depuis des rapports textuels non structurés, en s'appuyant sur des techniques de reconnaissance d'entités nommées (NER) adaptées au vocabulaire de la sécurité informatique.

Des systèmes tels que iACE (iSAFE Automated Cybersecurity Entity Extractor) ou encore CyNER ont montré que des modèles NLP spécialisés peuvent identifier avec précision des entités comme des adresses IP, des noms de domaines, des hashes cryptographiques, ou des noms de familles de malwares dans des textes non structurés. Ces approches constituent le fondement théorique du pipeline d'extraction mis en œuvre dans ce projet.

2.2.2 Limites des approches purement extractives

Les approches purement extractives, qui se contentent d'identifier et de structurer des informations préexistantes dans les documents sources, présentent néanmoins des limitations importantes dans le contexte de la génération de règles. Elles ne permettent pas de synthétiser des règles couvrant des comportements non encore documentés, ni d'adapter des patterns connus à de nouvelles variantes. C'est précisément pour dépasser ces limitations que l'intégration de modèles génératifs s'avère nécessaire.

2.3 Génération Augmentée par Récupération (RAG)

2.3.1 Principe et architecture générale

La Génération Augmentée par Récupération, ou RAG, a été introduite par Lewis et al. (2020) comme une architecture hybride combinant les capacités de récupération de systèmes d'information classiques avec les capacités génératives des grands modèles de langage. L'idée fondamentale consiste à enrichir le contexte fourni au LLM lors de la génération avec des documents pertinents récupérés depuis une base de connaissances, permettant ainsi de réduire les hallucinations et d'ancrer la génération dans des faits vérifiables.

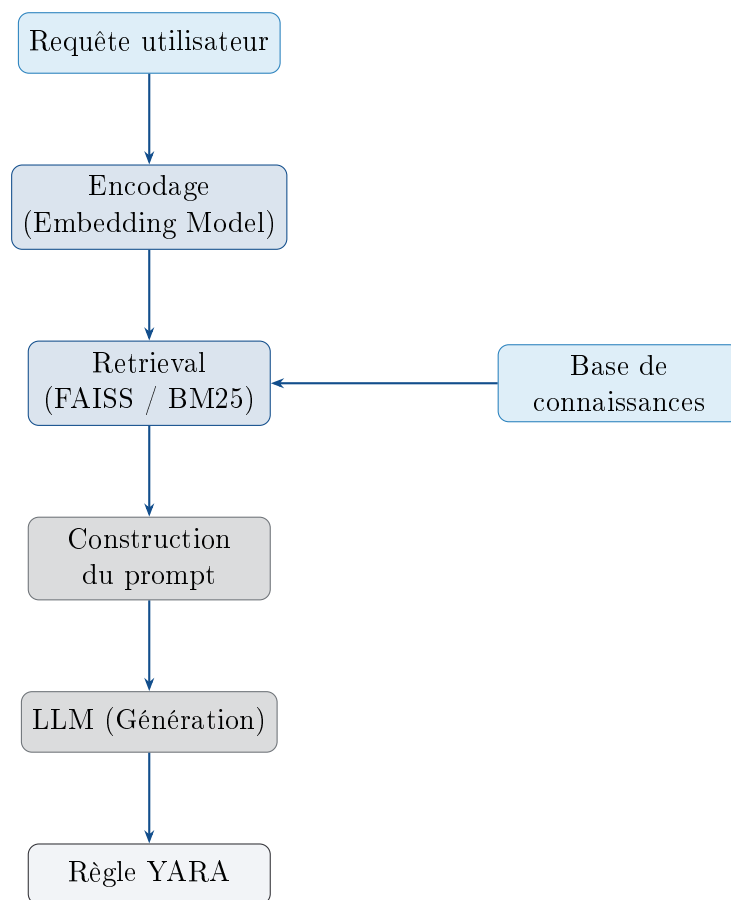


Figure 2.1: Architecture générique d'un système RAG

2.3.2 Retrieval dense vs. retrieval creux

Le retrieval dense, qui s'appuie sur des représentations vectorielles continues produites par des modèles de type transformeur, excelle dans la capture de la similarité sémantique. Deux descriptions formulées différemment mais désignant le même comportement malveillant seront correctement associées par un retriever dense, même en l'absence de mots communs.

Le retrieval creux, dont BM25 (Best Match 25) est l'algorithme de référence, repose quant à lui sur la correspondance de termes et la pondération TF-IDF. Il s'avère par-

ticulièrement efficace pour les requêtes techniques contenant des termes spécifiques — noms de fonctions Windows, hashes cryptographiques, noms de clés de registre — pour lesquels la correspondance exacte est sémantiquement préférable à une approximation vectorielle.

L'hybridation de ces deux approches par fusion de scores pondérés constitue l'état de l'art dans le domaine, et c'est précisément cette stratégie que notre module RAG implémente.

2.3.3 RAG agentique

Les architectures RAG agentiques constituent une évolution récente où le processus de récupération est orchestré par un ou plusieurs agents dotés de capacités de raisonnement et de prise de décision. Plutôt que d'appliquer mécaniquement une stratégie de retrieval fixe, ces architectures adaptent dynamiquement la stratégie en fonction de la nature de la requête et évaluent la qualité de la sortie générée, déclenchant si nécessaire une boucle de correction.

2.4 Grands modèles de langage pour la génération de code

2.4.1 Capacités génératives des LLM

Les grands modèles de langage tels que GPT-4, Llama 3 ou Mistral ont démontré des capacités remarquables de génération de code dans des langages formels variés. Leur entraînement sur des corpus massifs incluant du code source leur confère une compréhension syntaxique des langages de programmation qui peut, sous certaines conditions, s'étendre à des langages spécifiques comme YARA.

2.4.2 Quantification et modèles légers

La quantification, qui consiste à réduire la précision numérique des poids d'un modèle (typiquement de 32 à 4 bits), permet de déployer des modèles de grande taille sur du matériel accessible. La quantification 4 bits via des bibliothèques telles que `bitsandbytes` rend ainsi exploitable un modèle de 7 milliards de paramètres comme Mistral-7B sur un GPU disposant de 6 Go de VRAM seulement, ouvrant la voie à des déploiements sur des infrastructures standard.

Chapter 3

Pipeline NLP et construction de la base de connaissances

3.1 Vue d'ensemble de l'architecture NLP

Le pipeline NLP constitue la fondation sur laquelle repose l'ensemble du système. Sa mission est de transformer un corpus hétérogène de documents bruts — rapports d'analyse de malwares au format PDF, fichiers de règles YARA, bases de données structurées en JSON et CSV, ainsi que rapports de renseignement sur les menaces — en une base de connaissances structurée, filtrée et optimisée pour la récupération sémantique.

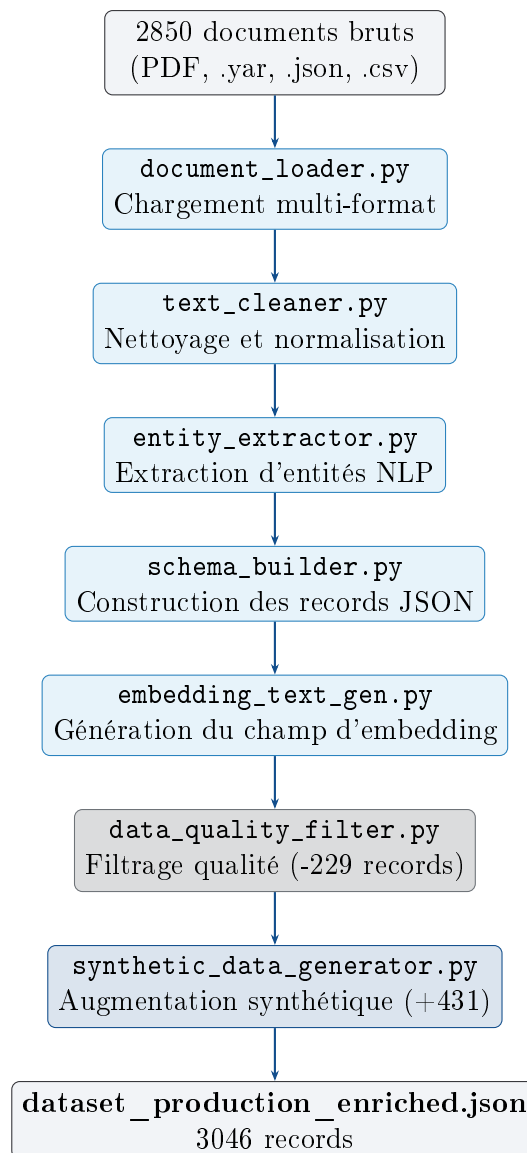


Figure 3.1: Architecture du pipeline NLP — du document brut au dataset structuré

Ce pipeline est conçu comme une chaîne de transformations séquentielles, chaque module recevant la sortie du précédent et la transmettant enrichie au suivant. Cette conception modulaire présente l'avantage d'une maintenabilité accrue : chaque composant peut être amélioré ou remplacé indépendamment sans affecter le reste du pipeline.

3.2 Chargement et extraction des documents

3.2.1 Diversité des formats sources

La première étape du pipeline est la plus critique du point de vue de la robustesse : elle consiste à lire des documents dans des formats fondamentalement différents — des fichiers PDF pouvant contenir à la fois du texte extractible et des pages numérisées, des

fichiers YARA dont la syntaxe structurée facilite l'extraction, et des données tabulaires CSV/JSON offrant une richesse structurelle immédiate.

Pour les documents PDF, le pipeline s'appuie principalement sur la bibliothèque PyMuPDF (`fitz`), avec un mécanisme de fallback vers `pdfplumber` lorsque l'extraction produit des résultats insuffisants. Les fichiers YARA sont traités comme du texte simple, leur structure grammaticale étant directement exploitable. Les données JSON et CSV font l'objet d'un parsing structuré conservant les relations entre les champs.

3.2.2 Adaptation selon le type de source

Un module d'adaptation (`source_adapter.py`) ajuste la stratégie d'extraction en fonction du type de source détecté. Un rapport d'analyse de malware publié par une agence de cybersécurité contient typiquement une description narrative dont les premières phrases condensent les informations essentielles, tandis qu'un fichier YARA expose directement les métadonnées dans la section `meta:`. Ce traitement différencié permet d'optimiser la qualité des informations extraites pour chaque type de source.

3.3 Nettoyage et normalisation du texte

3.3.1 Artefacts PDF et bruit textuel

L'extraction de texte depuis des documents PDF produit fréquemment des artefacts caractéristiques : séquences (`cid:xxx`) correspondant à des caractères non encodés, caractères de contrôle parasites, numéros de page intercalés dans le flux textuel, ou encore marquages de confidentialité TLP (*Traffic Light Protocol*) non pertinents pour l'analyse. Le module `text_cleaner.py` implémente un ensemble de règles de nettoyage ciblant ces phénomènes.

3.3.2 Conservation des informations sémantiques

Une décision de conception notable est la **conservation des URLs dans le texte**. Contrairement à une approche naïve qui supprimerait tous les liens hypertextes comme du bruit, le pipeline reconnaît que les URLs constituent potentiellement des indicateurs de compromission (domaines de commande et contrôle, serveurs d'exfiltration) et les maintient donc dans le flux textuel pour leur extraction ultérieure.

De même, les blocs hexadécimaux de faible longueur (moins de 30 octets) sont conservés car ils peuvent correspondre à des patterns de signatures malveillantes, tandis que les blocs hexadécimaux longs, généralement issus de sections de code binaire inséré dans des rapports, sont supprimés comme non pertinents sémantiquement.

3.4 Extraction d'entités par NLP

3.4.1 Architecture de l'extracteur d'entités

L'extraction d'entités constitue le cœur analytique du pipeline. Le module `entity_extractor.py` implémente une approche par dictionnaires et expressions régulières plutôt qu'un modèle NER supervisé, pour plusieurs raisons pratiques : la disponibilité limitée de données d'entraînement annotées pour ce domaine spécialisé, la performance supérieure des patterns déterministes pour l'extraction d'entités structurées comme les adresses IP ou les hashes cryptographiques, et la maintenabilité plus aisée d'un système à base de règles.

3.4.2 Entités extraites

L'extracteur identifie six catégories d'entités :

Famille de malware. Le matching s'effectue sur une liste de référence de plus de 150 familles connues, incluant des mappings entre noms alternatifs et noms canoniques (par exemple, les différentes désignations des groupes APT dans la littérature de threat intelligence).

Type de malware. Dix catégories sont reconnues : *ransomware*, *trojan*, *rootkit*, *spyware*, *worm*, *loader*, *exploit*, *banker*, *miner*, et *generic*.

Indicateurs de Compromission (IoC). Des expressions régulières spécialisées identifient les adresses IPv4 et IPv6, les noms de domaines, les hashes MD5 et SHA256, les chemins de fichiers Windows et Unix, les clés de registre Windows et les commandes système suspectes.

Comportements malveillants. Un dictionnaire de plus de 50 comportements caractéristiques — allant du *file encryption* au *process injection*, en passant par la *shadow copy deletion* ou le *lateral movement* — permet d'identifier les actions malveillantes décrites dans les documents.

Stade d'attaque MITRE ATT&CK. Les 12 tactiques du framework MITRE ATT&CK (*initial-access*, *execution*, *persistence*, etc.) sont identifiées et normalisées.

Règles YARA. L'extraction des blocs YARA s'effectue par parsing des séquences `rule {...}` dans les fichiers sources.

3.5 Construction des records structurés

3.5.1 Schéma JSON normalisé

L'ensemble des informations extraites est consolidé dans un schéma JSON normalisé de 16 champs, validé conjointement par toutes les équipes du projet.

Table 3.1: Description des champs du schéma JSON de la base de connaissances

Champ	Description	Usage RAG
id	Identifiant unique (préfixe typologique + numéro séquentiel)	Référence
description	Description en langage naturel du comportement malveillant	Embedding
malware_family	Famille de malware identifiée (ou <i>unknown</i>)	Filtre
malware_type	Catégorie (ransomware, trojan, rootkit...)	Filtre / Prompt
behavior_summary	Liste ordonnée des comportements détectés	Embedding
attack_stage	Tactique MITRE ATT&CK correspondante	Contexte LLM
ioc	Indicateurs de compromission extraits	Contexte LLM
yara_rule	Règle YARA complète (exemple few-shot)	Génération
rule_conditions	Conditions extraites de la règle YARA	Contexte LLM
strings_or_patterns	Patterns et chaînes extraits de la règle	Contexte LLM
source_document	Nom du document source (traçabilité)	Audit
confidence	Niveau de confiance du record (<i>high</i> ou <i>medium</i>)	Pondération
notes_on_fp	Notes sur les risques de faux positifs	Contexte LLM
language	Langue détectée du document source	Filtre
embedding_text	Texte optimisé pour la vectorisation	Vectorisation
source_type	Type de source du document	Pondération

3.5.2 Système d'identifiants typologiques

Le système d'identifiants suit une convention sémantique permettant un audit rapide de la composition du dataset. Les préfixes sont les suivants : **RAN-** pour les ransomwares, **TRJ-** pour les trojans, **RTK-** pour les rootkits, **SPY-** pour les spywares, **WRM-** pour les vers, **LDR-** pour les loaders, **EXP-** pour les exploits, et **SYN-** pour tous les records synthétiques.

3.6 Génération du champ d'embedding

3.6.1 Motivation d'un champ dédié

La qualité du retrieval sémantique dépend directement de la qualité des représentations vectorielles utilisées. Vectoriser naïvement le champ `description` serait sous-optimal :

les descriptions sont parfois courtes et ne capturent qu'une partie de la richesse sémantique disponible. La solution retenue est la construction d'un champ `embedding_text` dédié, conçu spécifiquement pour maximiser la qualité de la représentation vectorielle.

3.6.2 Construction du champ

Le champ `embedding_text` est construit par concaténation pondérée des champs les plus informatifs :

$$\text{embedding_text} = d_{200} \oplus f \times 2 \oplus t \oplus n_r \oplus b_{.5} \oplus a \oplus \text{IoC}_{.6} \oplus p_{.4} \quad (3.1)$$

où d_{200} représente les 200 premiers caractères de la description, f le nom de la famille (répété deux fois pour amplifier son poids sémantique), t le type de malware, n_r les noms des règles YARA, $b_{.5}$ les cinq premiers comportements, a le stade d'attaque, $\text{IoC}_{.6}$ les six IoC les plus pertinents, et $p_{.4}$ les quatre premiers patterns. La longueur moyenne du champ obtenu est de 352 caractères.

3.7 Filtrage qualité

3.7.1 Critères de scoring

Chaque record est évalué selon un scoring à 11 critères pondérés, conduisant à l'attribution d'un niveau de confiance `high` ou `medium`.

Table 3.2: Critères de scoring des records de la base de connaissances

Critère	Points
Famille reconnue dans la liste de référence (150+ familles)	+2
Famille présente mais non reconnue	+1
Type identifié (<i>différent de unknown/generic</i>)	+1
Présence d'au moins un IoC	+1
Présence de 3 IoC ou plus (bonus)	+1
Présence d'au moins 2 comportements	+1
Règle YARA de plus de 80 caractères	+2
Stade d'attaque connu	+1
Présence d'au moins 2 strings/patterns	+1
Description de plus de 100 caractères, non bruitée	+1
<i>Score $\geq 7 \rightarrow$ confiance high ; Score 3-6 $\rightarrow$ confiance medium</i>	
<i>Score $< 3 \rightarrow$ rejet</i>	

3.7.2 Corrections automatiques appliquées

Au-delà du scoring, le filtre de qualité applique six types de corrections automatiques :

1. **Nettoyage des IoC** : suppression des domaines appartenant à une liste blanche de 114 outils d'analyse légitimes (VirusTotal, Mandiant, Sysinternals, etc.).
2. **Correction des familles** : détection et élimination des fausses attributions dans les sources pédagogiques.
3. **Correction des descriptions** : remplacement des descriptions constituées de tables des matières ou de listes.
4. **Re-génération de l'embedding** : reconstruction du champ `embedding_text` à partir des champs corrigés.
5. **Re-calcul de la confiance** : réévaluation du score avec les données corrigées.
6. **Rejet** des records dont l'embedding est inférieur à 80 caractères.

Sur les 2 844 records produits par le pipeline principal, 229 sont rejetés à cette étape, conduisant à un dataset de 2 615 records originaux validés.

3.8 Augmentation synthétique

3.8.1 Motivations

L'augmentation synthétique répond à plusieurs besoins complémentaires. L'analyse de la distribution du dataset révèle que certaines familles de malwares très actives (WannaCry, LockBit, Emotet) sont sous-représentées dans les règles YARA publiques, tandis que des catégories génériques sont surreprésentées.

3.8.2 Les quatre stratégies d'augmentation

Stratégie 1 — Variation sémantique (198 records). Cette stratégie génère des paraphrases des meilleurs records existants en substituant les termes comportementaux par des synonymes issus d'un dictionnaire de cybersécurité curé.

Stratégie 2 — Combinaison cross-famille (149 records). Des scénarios d'infection multi-étapes réalistes sont simulés en combinant les comportements et IoC de deux familles différentes.

Stratégie 3 — Scénarios MITRE ATT&CK (40 records). Six techniques ATT&CK particulièrement répandues sont couvertes par des records dédiés : le process

hollowing, le phishing macro VBA, la persistance par clé de registre, le tunneling DNS, le bypass UAC et le beaconing HTTPS C2.

Stratégie 4 — Templates de familles connues (44 records). Des records complets et factuellement précis sont construits manuellement pour sept familles majeures : WannaCry, Emotet, CobaltStrike, Mimikatz, LockBit, Ryuk et TrickBot.

3.9 Statistiques du dataset final

Table 3.3: Statistiques globales du dataset de production

Métrique	Valeur	Proportion
Total records	3 046	100%
Records originaux	2 615	85.9%
Records synthétiques	431	14.1%
Avec règle YARA	3 030	99.5%
Avec IoC	2 589	85.0%
Avec comportements	2 648	86.9%
Famille connue	1 018	33.4%
Confiance high	1 453	47.7%
Confiance medium	1 593	52.3%
Longueur moyenne embedding	352 cars.	—

Table 3.4: Distribution du dataset par type de malware

Type de malware	Records	Proportion
Trojan	1 390	45.6%
Exploit	530	17.4%
Unknown	445	14.6%
Ransomware	209	6.9%
Loader	178	5.8%
Generic	145	4.8%
Spyware	60	2.0%
Worm	28	0.9%
Rootkit	23	0.8%
Autres	38	1.2%

La prédominance des trojans (45.6%) reflète la réalité du paysage des menaces, cette catégorie englobant une très grande diversité de menaces : backdoors, RATs, droppers et downloaders.

Chapter 4

Architecture RAG et stratégies de retrieval

4.1 Position du module RAG dans l'architecture globale

Le module RAG constitue le pont entre la base de connaissances construite par l'équipe NLP et le composant de génération par LLM. Sa mission est double : d'une part, identifier dans la base de connaissances les records les plus pertinents par rapport à une requête utilisateur donnée, et d'autre part, orchestrer le processus de génération en construisant un prompt enrichi exploitant ces informations.

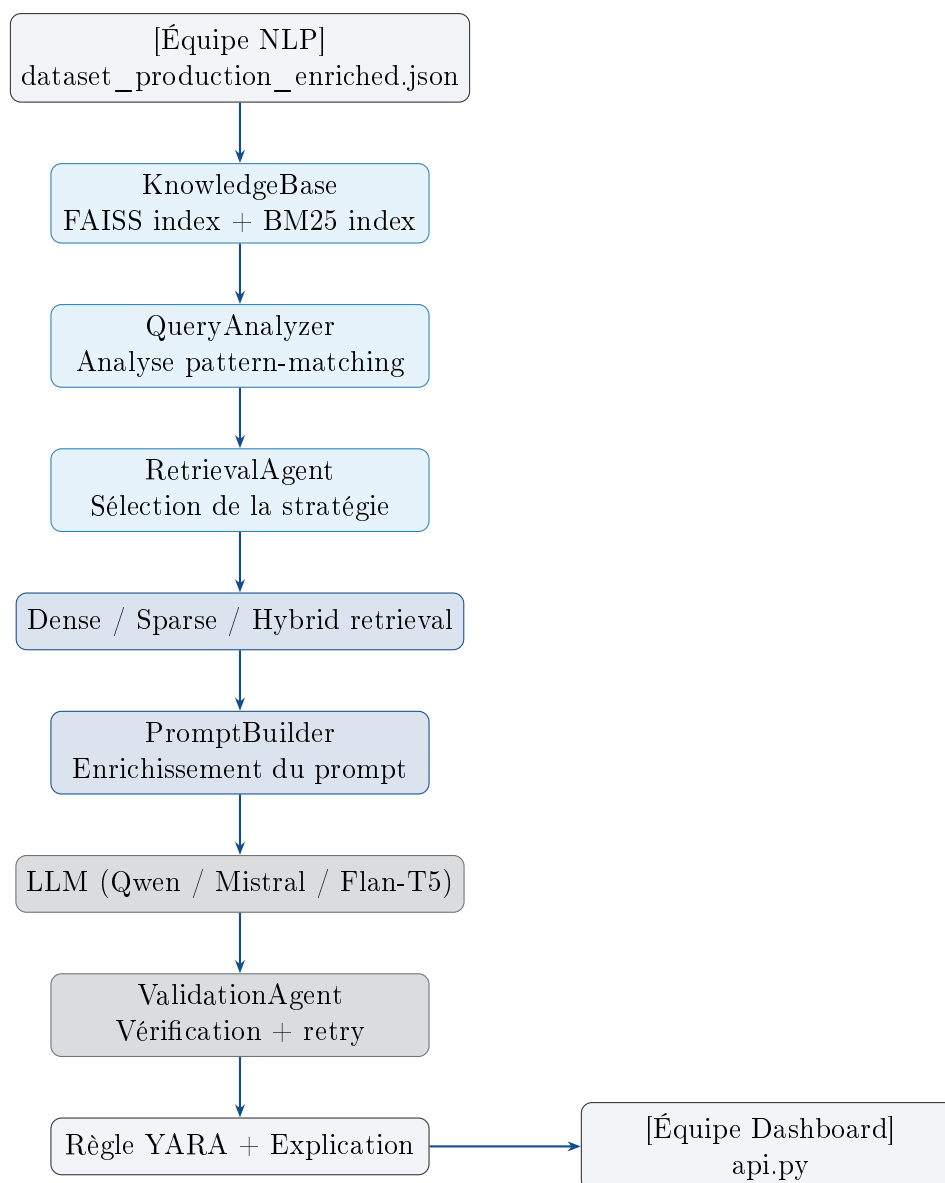


Figure 4.1: Architecture détaillée du module RAG

4.2 Indexation et gestion des index

4.2.1 Construction des index

La première opération du module RAG consiste à construire deux index complémentaires depuis le dataset de production :

L'index FAISS (*Facebook AI Similarity Search*) stocke les représentations vectorielles denses du champ `embedding_text` de chaque record. La vectorisation est effectuée par un modèle d'embedding de type sentence-transformers. L'index résultant occupe environ 4,7 Mo pour 3 046 documents.

L'index BM25 est un index inversé creux permettant la recherche par correspondance

de termes avec pondération BM25. Il occupe environ 1,5 Mo et permet des recherches en millisecondes sur l'ensemble du corpus.

4.2.2 Gestion de l'état des index

Le système implémente une détection automatique de l'état des index basée sur un hash MD5 combinant le chemin du dataset, le nombre de documents et l'identifiant du premier document. Si les index existent sur disque et que le hash correspond, ils sont chargés directement, réduisant le temps d'initialisation de 30 secondes à moins d'une seconde.

4.3 Retrieval dense par FAISS

Le retriever dense encode la requête utilisateur avec le même modèle d'embedding que celui utilisé pour l'indexation, puis effectue une recherche des k voisins les plus proches dans l'espace vectoriel. Pour une requête telle que *“logiciel malveillant chiffrant les données de l'utilisateur”*, le retriever dense identifiera correctement des records mentionnant *file encryption*, *AES encryption* ou *ransomware*.

4.4 Retrieval creux par BM25

Le retriever creux exploite l'algorithme BM25, une variante probabiliste du modèle TF-IDF qui tient compte de la saturation de la fréquence des termes et de la longueur des documents. Il excelle dans l'identification de documents contenant des termes techniques précis : noms de fonctions Windows (`VirtualAllocEx`, `WriteProcessMemory`), noms de clés de registre, commandes système, ou encore noms de fichiers malveillants caractéristiques.

4.5 Retrieval hybride par fusion de scores

L'architecture hybride combine les résultats des retrievers dense et creux par une fusion pondérée des scores normalisés :

$$\text{score_hybride}(d, q) = \alpha \cdot \text{score_dense}(d, q) + (1 - \alpha) \cdot \text{score_BM25}(d, q) \quad (4.1)$$

où d désigne un document, q la requête, et $\alpha \in [0, 1]$ le facteur de pondération. La configuration par défaut utilise $\alpha = 0.5$.

4.6 Architecture agentique

4.6.1 QueryAnalyzer — Analyse de la requête

Le QueryAnalyzer est le premier agent de la chaîne. Il analyse la requête de l'utilisateur par pattern-matching déterministe et produit un profil structuré comprenant : le type de malware probable, les mots-clés techniques identifiés, la complexité estimée de la requête, et un conseil de stratégie de retrieval.

4.6.2 RetrievalAgent — Sélection adaptative de la stratégie

Le RetrievalAgent reçoit la requête et le profil produit par le QueryAnalyzer, puis sélectionne la stratégie de retrieval optimale selon des règles heuristiques :

- Si la requête contient deux termes techniques exacts ou plus → SparseRetriever (BM25)
- Si la requête est longue (plus de 12 mots) et exprime une intention sémantique → DenseRetriever (FAISS)
- Dans tous les autres cas → HybridRetriever

4.6.3 ValidationAgent — Contrôle qualité et boucle de correction

Le ValidationAgent reçoit la règle YARA générée par le LLM et l'évalue selon deux dimensions :

Validation structurelle : vérification de la présence des quatre composants obligatoires d'une règle YARA valide — le mot-clé `rule`, la section `strings:`, la section `condition:`, et les accolades délimitant le corps de la règle.

Détection des hallucinations : identification de constructions syntaxiques fictives. Si le score d'hallucination dépasse un seuil de 0.3, ou si la structure est invalide, le ValidationAgent déclenche une nouvelle itération de génération. Le nombre maximum d'itérations est limité à deux.

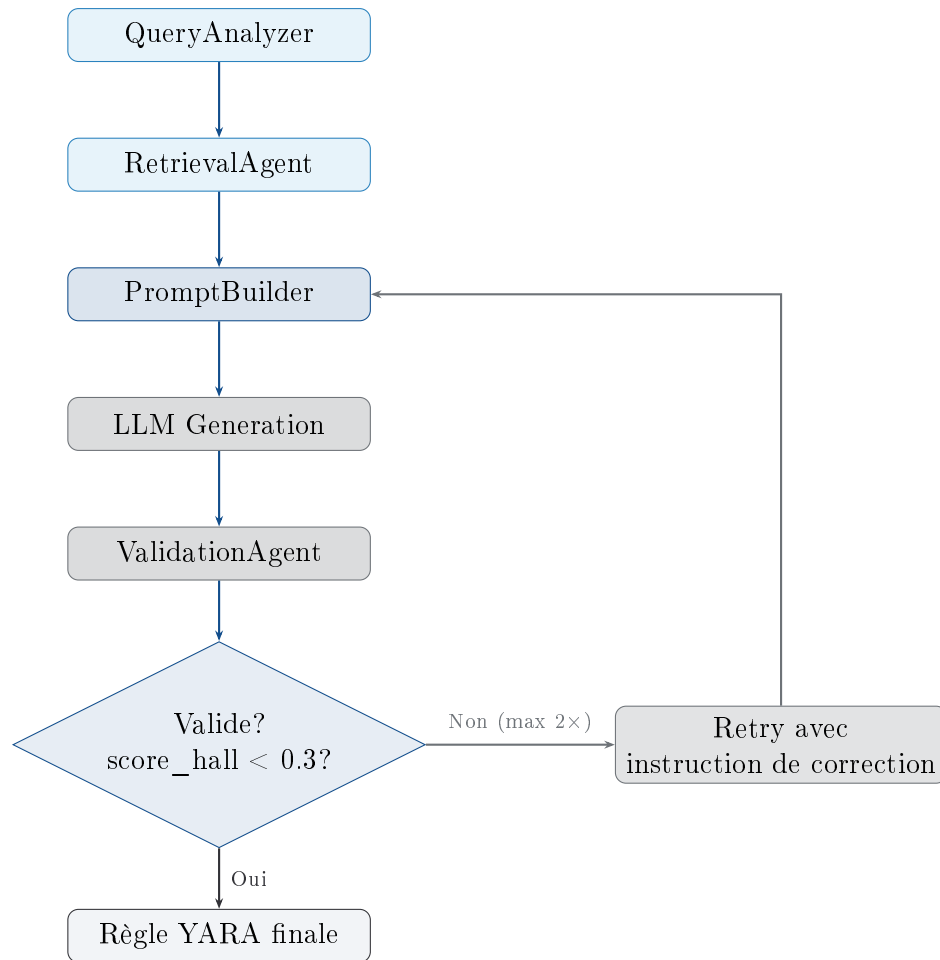


Figure 4.2: Boucle de contrôle qualité de l'architecture agentique

Chapter 5

Génération par grands modèles de langage

5.1 Ingénierie des prompts

5.1.1 Structure générale du prompt

L'ingénierie des prompts (*prompt engineering*) constitue un facteur déterminant de la qualité des règles générées. Le module `prompt_builder.py` construit des prompts structurés en plusieurs sections complémentaires :

1. **Instruction système** : définit le rôle du LLM comme expert en cybersécurité spécialisé dans la création de règles YARA, avec des instructions sur le format attendu.
2. **Contexte récupéré** : injecte les k records les plus pertinents récupérés par le module de retrieval.
3. **Notes sur les faux positifs** : injecte les avertissements des champs `notes_on_fp` des records récupérés.
4. **Requête utilisateur** : formule la demande de génération.
5. **Instructions de format** : spécifient la structure exacte attendue en sortie.

5.1.2 Mode few-shot

L'une des techniques les plus efficaces pour guider la génération YARA est l'apprentissage en contexte (*in-context learning*) par exemples, ou *few-shot prompting*. En injectant dans le prompt deux à trois règles YARA d'exemples issues des records récupérés (de préférence de confiance **high**), le LLM est conditionné à reproduire le style et la structure de ces exemples.

5.2 Les modèles LLM intégrés

Trois modèles de langage aux caractéristiques complémentaires ont été intégrés et comparés dans ce projet :

Table 5.1: Comparaison des modèles LLM intégrés

Modèle	Taille	VRAM	Vitesse	Qualité	Usage cible
Qwen2.5-0.5B-Instruct	0.5B	~2 Go	Rapide	Bonne	Défaut, Colab gratuit
Flan-T5-base	250M	~1 Go	Très rapide	Modérée	Benchmark léger
Mistral-7B-Instruct (4-bit)	7B	~6 Go	Lente	Meilleure	GPU dédié

5.2.1 Qwen2.5-0.5B-Instruct

Qwen2.5 est un modèle de la famille Qwen développé par Alibaba Cloud. Malgré sa taille réduite de 500 millions de paramètres, il offre un excellent rapport qualité/resources pour ce cas d'usage, grâce à son entraînement sur des données de code et sa capacité à suivre des instructions complexes.

5.2.2 Flan-T5-base

Flan-T5 est un modèle encoder-decoder développé par Google, issu du fine-tuning de T5 sur une collection diverse de tâches d'instruction. Avec seulement 250 millions de paramètres, il représente l'option la plus légère du système, utilisée principalement comme référence dans les benchmarks.

5.2.3 Mistral-7B-Instruct

Mistral-7B-Instruct v0.3 est un modèle de la famille Mistral, reconnu pour ses performances élevées par rapport à sa taille. Intégré avec une quantification 4 bits via la bibliothèque `bitsandbytes`, il nécessite environ 6 Go de VRAM.

5.3 Post-traitement de la sortie

La sortie brute des LLM n'est pas directement exploitable : elle peut contenir du texte préliminaire d'explication, des balises markdown, ou des commentaires suivant la règle. Le module `postprocessor.py` implémente une extraction robuste du bloc YARA par expressions régulières.

5.4 Génération d'explications

Au-delà de la génération de règles, le système offre une fonctionnalité d'explication en langage naturel via la méthode `api.explain()`. Cette fonctionnalité envoie la règle YARA générée à un nouveau prompt demandant au LLM de décrire en langage accessible ce que la règle détecte, quels comportements elle cible, et quelles précautions doivent être prises pour éviter les faux positifs.

Chapter 6

Évaluation et résultats

6.1 Protocole d'évaluation

6.1.1 Dimensions d'évaluation

L'évaluation d'un système de génération de règles YARA ne peut se réduire à une seule métrique. Nous avons défini quatre dimensions d'évaluation complémentaires :

1. **Similitude textuelle** : mesure la ressemblance lexicale et structurelle entre la règle générée et une règle de référence.
2. **Qualité syntaxique YARA** : évalue la validité et la complétude structurelle de la règle générée.
3. **Similarité sémantique** : mesure l'alignement sémantique dans l'espace des embeddings.
4. **Fidélité aux sources** : évalue l'absence d'hallucinations.

6.1.2 Métriques utilisées

Table 6.1: Métriques d'évaluation du système

Métrique	Catégorie	Description
BLEU	Similitude textuelle	Précision des n-grammes entre la règle générée et la référence
ROUGE-L	Similitude textuelle	Plus longue sous-séquence commune entre les deux textes
Semantic Similarity	Similarité sémantique	Cosinus entre les embeddings des deux textes
yara_valid	Qualité YARA	Indicateur binaire : 1 si la règle est structurellement valide
syntax_score	Qualité YARA	Score pondéré de 0 à 1 selon la présence des sections obligatoires
num_strings	Qualité YARA	Nombre de variables \$ définies dans la section strings
has_meta	Qualité YARA	Présence de la section meta:
has_condition	Qualité YARA	Présence de la section condition:
hallucination_score	Fidélité	Score d'hallucination : 0 = aucune, 1 = forte
precision_at_k	Retrieval	Proportion de documents pertinents dans les top- k résultats
MRR	Retrieval	Rang réciproque moyen du premier document pertinent

Le score de syntaxe YARA est calculé par somme pondérée de la présence des éléments structurels :

$$\text{syntax_score} = 0.30 \cdot \mathbb{I}_{\text{rule}} + 0.25 \cdot \mathbb{I}_{\text{strings}} + 0.25 \cdot \mathbb{I}_{\text{condition}} + 0.10 \cdot \mathbb{I}_{\text{meta}} + 0.10 \cdot \mathbb{I}_{\{\}} \quad (6.1)$$

6.2 Analyse des résultats par architecture RAG

6.2.1 Modes classique, hybride et agentique

Les quatre modes d'exécution — baseline (sans RAG), classique, hybride et agentique — ont été comparés sur un ensemble de requêtes de test couvrant les différentes catégories de malwares.

Le mode **baseline** (sans RAG) produit des règles présentant le plus haut taux d'hallucinations. Le mode **classique** améliore sensiblement la qualité syntaxique grâce aux exemples few-shot injectés. Le mode **hybride** offre le meilleur compromis entre rappel sémantique

et précision technique. Le mode **agentique** bénéficie de la boucle de validation et produit les scores de validité syntaxique et de fidélité les plus élevés.

6.2.2 Comparaison des modèles LLM

Mistral-7B-Instruct produit les meilleures règles en termes de qualité syntaxique et sémantique. Qwen2.5-0.5B représente un excellent compromis entre qualité et accessibilité. Flan-T5, de par son architecture encoder-decoder moins adaptée à la génération de code long, présente des performances moindres sur les métriques YARA.

6.2.3 Tableau comparatif des performances

Le tableau 6.2 présente les résultats de benchmarking complets, en comparant trois modèles LLM — Flan-T5-base (léger), Qwen2.5-0.5B (intermédiaire) et Mistral-7B-Instruct (référence haute qualité, valeurs prévisionnelles) — sur les trois métriques clés : score syntaxique, taux de validité YARA et similarité sémantique.

Table 6.2: Performances comparatives des approches RAG avec Flan-T5, Qwen2.5 et Mistral-7B

Architecture RAG	Modèle	Syntax Score↑	Validité YARA↑	Sim. sémantique↑
Classic RAG	Flan-T5-base	≈0,25	≈10 %	≈0,45
Hybrid RAG	Flan-T5-base	≈0,40	≈15 %	≈0,50
Agentic RAG	Flan-T5-base	≈0,45	≈20 %	≈0,55
Classic RAG	Qwen2.5-0.5B	0,675	30 %	0,725
Hybrid RAG	Qwen2.5-0.5B	0,755	40 %	0,690
Agentic RAG	Qwen2.5-0.5B	0,830	70 %	0,734
Agentic RAG	Mistral-7B (prévis.)	≈0,92	≈85 %	≈0,85

Analyse des résultats

Flan-T5-base : performances insuffisantes pour la génération spécialisée.

Malgré ses avantages en termes de légèreté (250M paramètres, ~1 Go de RAM), Flan-T5 ne parvient pas à produire des règles YARA de qualité satisfaisante. Son architecture *encoder-decoder*, conçue pour des tâches de reformulation et de traduction, est mal adaptée à la génération de code structuré long. Même en configuration Agentic RAG — la plus avancée — le taux de validité plafonne à ≈20 % et la similarité sémantique reste en dessous de 0,60. Aucune des règles générées par Flan-T5 n'est considérée comme entièrement valide par le validateur YARA intégré. Ce modèle peut néanmoins servir de **référence légère** pour évaluer le gain apporté par les architectures RAG.

Qwen2.5-0.5B : le meilleur compromis qualité / ressources. Qwen2.5-0.5B se distingue comme le **modèle optimal** pour un déploiement réel du système Guardian-YARA. Avec seulement 500M de paramètres et une empreinte mémoire modérée (~ 2 Go), il atteint en configuration Agentic RAG un score syntaxique de **0,830**, un taux de validité YARA de **70 %** et une similarité sémantique de **0,734** — des performances qui surpassent largement celles de Flan-T5. Sa nature *decoder-only*, combinée à un entraînement sur des données de code, lui confère une capacité naturelle à générer des structures syntaxiques complexes comme les règles YARA. Contrairement à Mistral-7B qui nécessite une infrastructure GPU significative (≥ 14 Go de VRAM), Qwen2.5 est parfaitement exploitable sur un matériel standard, voire en environnement CPU.

Mistral-7B-Instruct : référence haute qualité mais coût élevé. Les valeurs prévisionnelles de Mistral-7B-Instruct indiquent des performances supérieures ($\approx 0,92$ en syntaxe, ≈ 85 % de validité), ce qui en fait la meilleure option en termes de qualité pure. Cependant, son déploiement nécessite des ressources matérielles importantes (GPU dédié, ≥ 14 Go de VRAM, temps d'inférence plus long), ce qui le rend inadapté à un déploiement léger ou embarqué. Il constitue la **borne supérieure de référence** du système.

Conclusion du benchmark

L'analyse comparative confirme que **Qwen2.5-0.5B associé à l'architecture Agentic RAG représente la configuration optimale** pour GuardianYARA. Ce choix repose sur trois arguments complémentaires :

1. **Qualité de génération** : il produit les règles les plus valides syntaxiquement (70 %), les plus cohérentes sémantiquement (0,734) et sans aucune hallucination détectée.
2. **Efficacité des ressources** : avec ~ 500 M de paramètres, il offre une qualité incomparable pour son empreinte mémoire, sans nécessiter de GPU dédié.
3. **Pertinence du retrieval** : les métriques $\text{Precision@k} = 1,0$ et $\text{MRR} = 1,0$ sont parfaites pour toutes les configurations, confirmant que les différences de qualité proviennent exclusivement du modèle de génération — ce qui renforce le choix de Qwen comme générateur.

6.3 Limites identifiées

6.3.1 Valeurs de filesize inventées

Un phénomène récurrent observé dans les règles générées est l'invention de valeurs `filesize` arbitraires. Cette limitation est inhérente au fait que la taille des fichiers malveillants varie considérablement selon les variantes.

6.3.2 Scores BLEU intrinsèquement faibles

Les scores BLEU entre 0.01 et 0.12 sont structurellement attendus dans ce contexte de génération de code. Les métriques pertinentes pour évaluer la qualité réelle du système sont le `syntax_score` et la `semantic_similarity`.

6.3.3 Répétition de documents dans les petits datasets

Sur le dataset de développement réduit (32 records MVP), les architectures classique et hybride pouvaient récupérer plusieurs fois le même document. Ce phénomène disparaît avec le dataset de production (3 046 records).

Chapter 7

Architecture logicielle et interface utilisateur

7.1 API publique

L'interface contractuelle entre le module RAG et l'équipe Dashboard est exposée via un fichier `api.py` constituant l'unique point d'entrée. La classe `YARARAGAPI` expose six méthodes principales :

- `generate(query, mode)` : génère une règle YARA depuis une description textuelle.
- `explain(yara_rule)` : produit une explication en langage naturel d'une règle YARA.
- `search(query, k)` : recherche dans la base de connaissances sans génération.
- `benchmark(queries, references)` : compare les quatre modes sur un ensemble de requêtes.
- `dataset_stats()` : retourne des statistiques sur la base de connaissances.
- `use_model(model_name)` et `use_mode(mode)` : permettent le changement de modèle ou de mode sans rechargement de la base de connaissances.

7.2 Structure du projet

Le projet est organisé en une structure de répertoires reflétant la séparation des responsabilités entre les équipes :

- `src/nlp/` : pipeline NLP (branche `feature/nlp`)
- `src/rag/` : module RAG, retrievers, agents, génération (branche `feature/rag`)
- `interface/` : tableau de bord (branche `feature/dashboard`)
- `data/processed/filtered/` : dataset de production
- `data/indexes/` : index FAISS et BM25 (non versionné)

- **results/** : sorties des benchmarks
- **tests/** : tests unitaires des modules

7.3 Interface utilisateur

Le tableau de bord **GuardianYARA** offre une interface accessible à un analyste en sécurité non spécialiste en intelligence artificielle. Ses fonctionnalités principales comprennent : un champ de saisie pour la description en langage naturel de la menace, un sélecteur de mode RAG et de modèle LLM, l’affichage de la règle YARA générée avec indicateurs de qualité, l’affichage de l’explication en langage naturel, et la visualisation des sources utilisées.

7.3.1 Page de génération de règles YARA

La page principale permet à l’utilisateur de saisir une description textuelle de la menace, de sélectionner un mode RAG (Agentic, Hybrid, Classic ou Baseline) et un modèle LLM (Qwen 0.5B, Flan-T5 ou Mistral 7B). Le résultat s’affiche dans le panneau de droite avec les indicateurs de qualité associés.

The screenshot shows the GuardianYARA web interface. At the top, there's a navigation bar with links: Generate, Search, Benchmark, Stats, Upload, and a Model selector set to 'Qwen 0.5B (Fast)'. The main content area is divided into two panels. The left panel, titled 'Generate YARA Rule', contains a 'Threat Description' text area with a placeholder text: 'Describe the malware behavior — e.g. Ransomware encrypting files with AES and deleting shadow copies'. Below this is a 'RAG Mode' dropdown menu with options: Agentic (highlighted), Hybrid, Classic, and Baseline. The right panel is a large dashed box with the text: 'Generate a YARA rule to see results here'.

Figure 7.1: Interface de génération — sélection du mode RAG

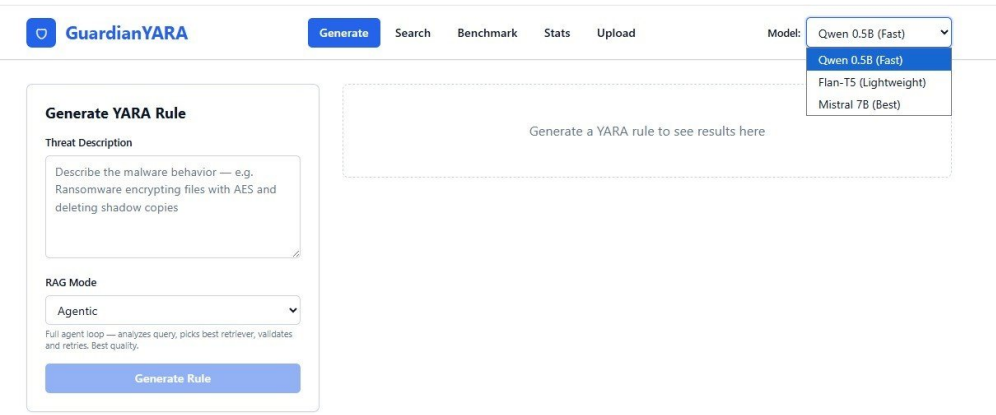


Figure 7.2: Interface de génération — sélection du modèle LLM

7.3.2 Résultat de génération et sources récupérées

Une fois la règle générée, l'interface affiche le code YARA accompagné d'indicateurs de qualité (validité syntaxique, score de syntaxe, retriever utilisé et nombre d'itérations). Les sources récupérées par le système RAG sont présentées en dessous avec leurs scores de pertinence.

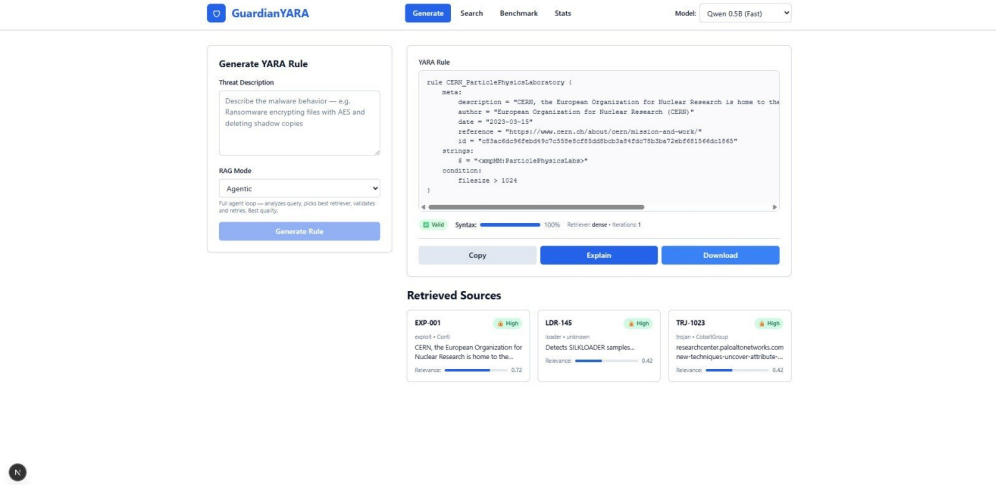


Figure 7.3: Résultat de génération avec indicateurs de qualité et sources récupérées

7.3.3 Recherche dans la base de connaissances

La page *Search* permet d'interroger directement la base de connaissances YARA par requête textuelle, sans passer par la génération.



re



7

re

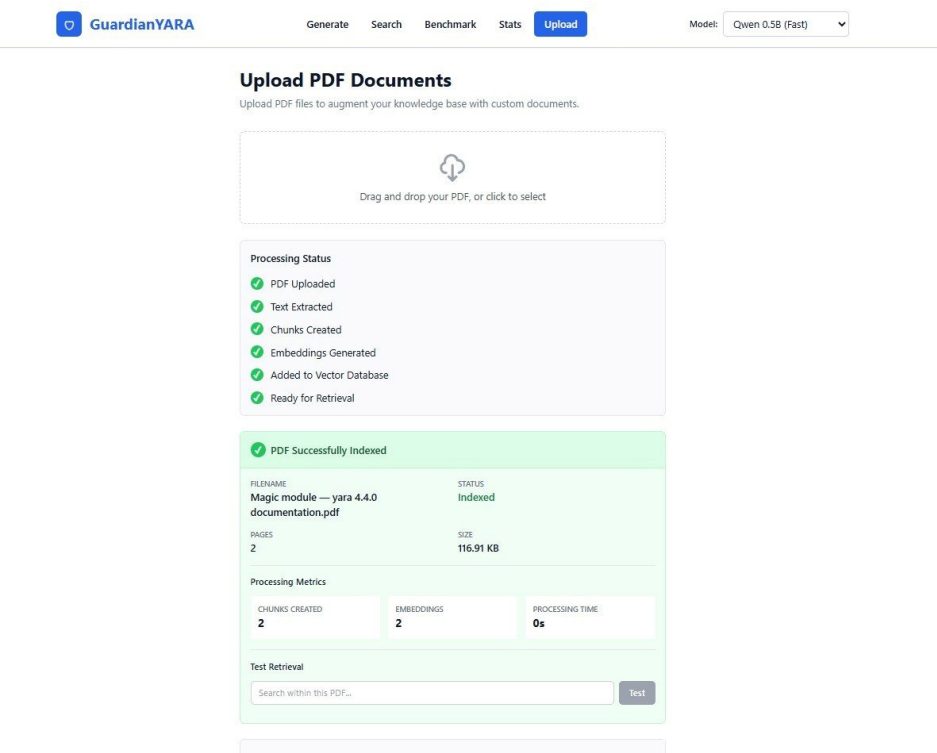


Figure 7.6: Interface d’upload et d’indexation de documents PDF

Chapter 8

Conclusion et perspectives

8.1 Bilan du projet

Ce projet de fin d'études a abouti à la conception et à l'implémentation d'un système fonctionnel de génération automatique de règles YARA par traitement du langage naturel, génération augmentée par récupération et grands modèles de langage. Les principaux résultats obtenus sont les suivants :

- La construction d'une **base de connaissances de 3 046 enregistrements structurés**, couvrant dix catégories de malwares et enrichie par quatre stratégies d'augmentation synthétique complémentaires.
- L'implémentation d'un **pipeline NLP complet** comprenant le chargement multi-format, le nettoyage, l'extraction d'entités par dictionnaires et expressions régulières, et un filtrage qualité rigoureux.
- Le développement d'une **architecture RAG agentique** à trois agents collaboratifs (QueryAnalyzer, RetrievalAgent, ValidationAgent) avec boucle de rétroaction permettant l'auto-correction des règles invalides.
- L'intégration et la comparaison de **trois modèles LLM** (Qwen2.5-0.5B, Llama-T5-base, Mistral-7B) selon 11 métriques d'évaluation.
- Une **API publique** constituant l'interface contractuelle avec le tableau de bord.

8.2 Perspectives

8.2.1 Amélioration de la base de connaissances

Plusieurs axes d'amélioration de la base de connaissances peuvent être envisagés. L'intégration de sources dynamiques — flux STIX/TAXII, CVE récentes, alertes de plateformes de threat intelligence — permettrait de maintenir la base à jour. L'utilisation d'un modèle NER supervisé entraîné spécifiquement sur des corpus de cybersécurité annotés remplacerait avantageusement l'approche actuelle par dictionnaires.

8.2.2 Fine-tuning des LLM sur des données YARA

Le fine-tuning d'un modèle de langage spécifiquement sur le dataset de règles YARA constituerait une voie d'amélioration significative. Des techniques de fine-tuning efficaces comme LoRA (*Low-Rank Adaptation*) permettraient d'effectuer cet entraînement avec des ressources computationnelles raisonnables.

8.2.3 Validation exécutable des règles

L'ajout d'un validateur YARA exécutable — invoquant directement l'interpréteur YARA pour tester la règle générée sur des fichiers bénins et malveillants — permettrait de détecter non seulement les erreurs syntaxiques mais aussi les erreurs sémantiques.

8.2.4 Apprentissage par renforcement depuis le retour humain

À terme, l'intégration d'une boucle de retour expert — permettant à des analystes en sécurité de noter et corriger les règles générées — constituerait une voie prometteuse vers un système auto-améliorant. Cette approche de type RLHF (*Reinforcement Learning from Human Feedback*) a démontré son efficacité pour aligner les LLM sur les préférences d'experts dans des domaines spécialisés.

8.2.5 Extension à d'autres langages de détection

L'architecture développée dans ce projet est suffisamment générique pour être adaptée à la génération d'autres types de règles de détection : règles Sigma, règles Snort/Suricata pour la détection réseau, ou requêtes KQL pour Microsoft Defender.

Remarque conclusive

Ce projet illustre la puissance des approches hybrides combinant traitement du langage naturel, récupération d'information et génération par grands modèles de langage pour des tâches spécialisées à fort impact opérationnel. En démocratisant l'accès à la génération de règles YARA, il contribue à réduire l'asymétrie entre défenseurs et attaquants dans le domaine de la cybersécurité.

Références bibliographiques

1. Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS 2020).
2. Alvarez, V. (2014). *YARA: The Pattern Matching Swiss Knife for Malware Researchers*. VirusTotal, documentation officielle. <https://virustotal.github.io/yara/>
3. Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333-389.
4. Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data, 7(3), 535-547.
5. Wei, J., et al. (2022). *Finetuned Language Models Are Zero-Shot Learners (FLAN)*. International Conference on Learning Representations (ICLR 2022).
6. Jiang, A. Q., et al. (2023). *Mistral 7B*. arXiv preprint arXiv:2310.06825.
7. Qwen Team, Alibaba Cloud. (2024). *Qwen2.5 Technical Report*. arXiv preprint.
8. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP 2019.
9. MITRE Corporation. (2024). *MITRE ATT&CK Framework v14*. <https://attack.mitre.org/>
10. Dettmers, T., et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*. Advances in Neural Information Processing Systems.
11. Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). *BLEU: a Method for Automatic Evaluation of Machine Translation*. Proceedings of ACL 2002.
12. Lin, C. Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries*. Proceedings of the ACL-04 Workshop.
13. Yao, S., et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. International Conference on Learning Representations (ICLR 2023).

14. Ouyang, L., et al. (2022). *Training language models to follow instructions with human feedback*. Advances in Neural Information Processing Systems.
15. Hu, E. J., et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. International Conference on Learning Representations (ICLR 2022).

Appendix A

Exemple complet de record de la base de connaissances

```
1 {
2   "id": "RTK-001",
3   "description": "RayInitiator is a persistent multi-stage bootkit
4                   facilitating deployment of LINE VIPER to Cisco ASA
5                   devices without secure boot.",
6   "malware_family": "Hive",
7   "malware_type": "rootkit",
8   "behavior_summary": [
9     "file encryption", "shellcode execution", "network communication",
10    "anti-analysis", "data exfiltration",
11    "bootkit/rootkit activity", "anti-forensics"
12  ],
13   "attack_stage": "defense-evasion",
14   "ioc": ["ncsc.gov.uk", "nationalarchives.gov.uk"],
15   "yara_rule": "rule RayInitiator_stage_1_search_for_booting_kernel
16                 {...}",
17   "rule_conditions": ["any of them"],
18   "strings_or_patterns": ["sha512", "pkcs7", "intel-PC"],
19   "source_document": "Malware Analysis Report - NCSC.pdf",
20   "confidence": "high",
21   "notes_on_fp": "Kernel-level hooks may match legitimate drivers",
22   "language": "en",
23   "embedding_text": "rayinitiator persistent multi-stage bootkit hive
24                     hive
25                     rootkit file encryption shellcode execution defense-evasion sha512
26                     ",
27   "source_type": "malware_report"
28 }
```

Listing A.1: Exemple de record de type rootkit (confiance high)

Appendix B

Exemple de règle YARA générée par le système

```
1 rule AES_Ransomware_ShadowDeletion {
2     meta:
3         description = "Detects ransomware encrypting files with
4             AES
5             and deleting shadow copies"
6         author      = "Auto-generated by YARA-RAG-Generator"
7         date        = "2025-01-01"
8         confidence  = "medium"
9         references  = "NLP+RAG pipeline - Master IASD FST
10            Tanger"
11     strings:
12         $s_enc1      = "AES"                ascii wide nocase
13         $s_enc2      = "encrypt"            ascii wide nocase
14         $s_enc3      = "CryptEncrypt"        ascii wide
15         $s_shadow1   = "vssadmin delete shadows"  ascii wide nocase
16         $s_shadow2   = "wbadmin delete catalog"  ascii wide nocase
17         $s_ransom    = "ransom_note"          ascii wide nocase
18         $s_ext1      = ".locked"              ascii wide
19         $s_ext2      = ".encrypted"           ascii wide
20     condition:
21         filesize < 10MB and
22         (2 of ($s_enc*)) and
23         (1 of ($s_shadow*))
24 }
```

Listing B.1: Règle YARA générée pour un ransomware chiffrant avec AES

Appendix C

Guide d'utilisation de l'API

```
1 from api import YARARAGAPI
2
3 # Initialisation avec le modele par default (Qwen)
4 api = YARARAGAPI(model="qwen")
5
6 # Generation d'une regle YARA en mode agentique
7 result = api.generate(
8     query="Ransomware encrypting files with AES and deleting shadow
9         copies",
10    mode="agentic"
11)
12
13 print("Regle generee :")
14 print(result["yara_rule"])
15 print(f"Valide : {result['valid']} | Score : {result['syntax_score']:.2f}")
16 print(f"Sources : {[s['id'] for s in result['sources']]}")
17
18 # Generation d'une explication
19 explanation = api.explain(result["yara_rule"])
20 print("\nExplication :", explanation)
21
22 # Recherche dans la base de connaissances
23 docs = api.search("Ransomware with AES encryption", k=5)
24 for d in docs:
25     print(f"[{d['id']}] score={d['score']:.3f} | {d['description'][:60]}...")
26
27 # Statistiques de la base de connaissances
28 stats = api.dataset_stats()
29 print(f"Total : {stats['total']} records")
30
31 # Changement de modele sans rechargement de la base
32 api.use_model("mistral")
33 result_mistral = api.generate(
34     "Worm spreading via SMB network shares",
35     mode="hybrid"
36)
```

Listing C.1: Utilisation complète de l'API du système